

Advanced R for Econometricians (Summer 2026)

Building GenAI Applications with R — Notes and Examples

Martin Arnold, Martin Schmelzer

How to Use These Notes

- Headers mirror slide titles for one-to-one lookup while teaching.
- Exercises are followed by a dedicated Solution subsection for fast reference in print.

Today: Build Small AI Tools in R

- This chapter builds reusable AI workflows in R: prompting, multi-turn context, OCR, structured extraction, and simple app assembly.
- Teaching rhythm per block: concept, live code, exercise, failure-mode discussion.

The Core Mental Model

- LLM usage in R is a remote function-call pattern with inputs (`model`, system prompt, user message, parameters) and outputs (text or structured objects).
- The objective is reliable language-task automation, not ad-hoc chat.

API Call – How R Talks to an LLM

- R sends HTTP/JSON requests to provider APIs and receives machine-readable responses.
- The API layer is language-agnostic; only client libraries differ across R/Python/Julia.

Why Groq + ellmer?

- Groq offers fast hosted inference and beginner-friendly access.
- `ellmer` provides a unified interface across providers with chat state, streaming, image inputs, and structured extraction.

Setup Checklist

- Install package: `install.packages("ellmer")`.
- Create and load key via `.Renviron: GROQ_API_KEY=...`
- Restart R session after editing `.Renviron`.
- For browser chat demos, install `shinychat`.

```
install.packages("ellmer")
usethis::edit_r_environ()

key <- Sys.getenv("GROQ_API_KEY")
```

```
nzchar(key)
substr(key, 1, 8)
```

The Minimal Pattern

- Core sequence: load `ellmer`, build `chat_groq()`, call `$chat()`, capture or print result.
- Missing API key is the first expected failure mode.

```
library(ellmer)

chat <- chat_groq(
  model = "llama-3.3-70b-versatile",
  system_prompt = "You are a helpful assistant."
)

chat$chat("What is R in one sentence?")
```

Exercise: One Prompt, Three Terms

Solution

Key idea: the system prompt is a persistent contract; only user messages vary.

```
library(ellmer)

chat <- chat_groq(
  model = "llama-3.3-70b-versatile",
  system_prompt = paste(
    "You are a concise econometrics tutor.",
    "For every term the user sends, respond with exactly two lines:",
    "Line 1: a one-sentence definition.",
    "Line 2: one concrete empirical example."
  ),
  echo = "none"
)

chat$chat("heteroskedasticity")
chat$chat("endogeneity")
chat$chat("multicollinearity")
```

The Chat Object Remembers

- Chat objects store turns, metadata, token usage, and cost information.
- Follow-up quality depends on history, but long history increases token/cost footprint.
- `token_usage()` is useful for monitoring cumulative usage.

```
chat <- chat_groq(
  model = "llama-3.3-70b-versatile",
  system_prompt = "You are a helpful R tutor."
)

chat$chat("What is a data frame?")
print(chat)
token_usage(chat)
```

Streaming vs. Capturing Output

- Interactive demos may stream responses.
- Scripts/functions should usually set `echo = "none"` and assign output.

```
chat <- chat_groq(  
  model = "llama-3.3-70b-versatile",  
  system_prompt = "Be concise.",  
  echo = "none"  
)  
  
response <- chat$chat("What is GDP?")  
cat(response)  
nchar(response)
```

Exercise: Capture and Inspect

Solution

```
chat <- chat_groq(  
  model = "llama-3.3-70b-versatile",  
  system_prompt = "You are a precise econometrics tutor.",  
  echo = "none"  
)  
  
answer <- chat$chat("Define heteroskedasticity in one sentence.")  
cat(answer)  
nchar(answer)
```

System Prompts Shape Behavior

- Keep question fixed and vary only system prompt to isolate behavior effects.
- Useful dimensions: role, audience, format, style constraints.

```
q <- "What is OLS regression?"  
  
chat1 <- chat_groq(  
  system_prompt = "Explain like I am 5 years old.",  
  echo = "none"  
)  
chat1$chat(q)  
  
chat2 <- chat_groq(  
  system_prompt = "Answer in exactly one sentence.",  
  echo = "none"  
)  
chat2$chat(q)  
  
chat3 <- chat_groq(  
  system_prompt = "You are a strict econometrics professor. Be formal and precise.",  
  echo = "none"
```

```
)
chat3$chat(q)
```

Make Prompt Experiments Reusable

```
ask <- function(system_message, user_question) {
  chat <- chat_groq(
    model = "llama-3.3-70b-versatile",
    system_prompt = system_message,
    echo = "none"
  )
  chat$chat(user_question)
}
```

- This helper makes prompt A/B tests reproducible and compact.

Multi-Turn Conversations

- `get_turns(include_system_prompt = TRUE)` is the recommended way to inspect/log accumulated context.

```
chat <- chat_groq(
  model = "llama-3.3-70b-versatile",
  system_prompt = "You are a helpful R tutor."
)

chat$chat("What is a factor in R?")
chat$chat("Can you give me an example?")
chat$chat("How is it different from a character vector?")

print(chat)
turns <- chat$get_turns(include_system_prompt = TRUE)
length(turns)
turns[[1]]
```

Exercise: Context Matters

Solution

```
chat <- chat_groq(
  model = "llama-3.3-70b-versatile",
  system_prompt = "You are an econometrics tutor. Be concise."
)

chat$chat("What is endogeneity?")
chat$chat("Give me an example from labor economics.")
chat$chat("How would I test for it?")

print(chat)
turns <- chat$get_turns(include_system_prompt = TRUE)
length(turns)
```

Interactive Chat

- `live_console()` and `live_browser()` are exploration interfaces.
- Keep production workflows in callable functions around `$chat()`.

```
chat <- chat_groq(
  model = "llama-3.3-70b-versatile",
  system_prompt = "You are a friendly R tutor."
)

live_console(chat)

install.packages("shinychat")

chat <- chat_groq(
  model = "llama-3.3-70b-versatile",
  system_prompt = "You are a data science assistant."
)

live_browser(chat)
```

Multimodal Models

- Vision-capable models accept text plus images and are useful for scanned forms, screenshots, handwritten notes, and embedded tables.
- For Groq image input in this deck, use `meta-llama/llama-4-scout-17b-16e-instruct`.

Document Scan to Markdown

- Use a vision-capable model and `content_image_file()`.
- Keep OCR prompts strict and deterministic (`temperature = 0`).
- Treat extracted markdown as a draft requiring review.

```
library(ellmer)

cwd <- getwd()
document_path <- paste0(cwd, "/SoSe_2026/AI/med_invoice.webp")
chat <- chat_groq(
  model = "meta-llama/llama-4-scout-17b-16e-instruct",
  system_prompt = paste(
    "You are a careful OCR assistant.",
    "Extract the visible document content and return only Markdown.",
    "Preserve headings, paragraphs, lists, tables, and signatures.",
    "Mark unreadable text as [illegible].",
  ),
  echo = "none",
  params = params(temperature = 0)
)
markdown_doc <- chat$chat(
  "Convert this scanned document into clean Markdown and translate to English.",
  content_image_file(document_path, resize = "high")
)
cat(markdown_doc)
```

- If magick is unavailable, install it or use `resize = "none"` for pre-sized images.

Exercise: Prompt the OCR

Solution

```
ocr_prompt <- paste(
  "You are a careful OCR assistant.",
  "Return only Markdown.",
  "Preserve headings, paragraphs, lists, and tables.",
  "Mark uncertain text as [unclear].",
  "End with a section named ## Extraction Notes that briefly lists any quality issues."
)

chat <- chat_groq(
  model = "meta-llama/llama-4-scout-17b-16e-instruct",
  system_prompt = ocr_prompt,
  echo = "none",
  params = params(temperature = 0)
)
```

Parameters and Providers

- Section divider matching the slide deck sequence.

Temperature

- Lower values maximize consistency and are preferred for extraction/study tooling.
- Higher values increase variability and are better for brainstorming.

```
low <- chat_groq(
  echo = "none",
  params = params(temperature = 0)
)
low$chat("Write a haiku about coding.")
low$chat("Write a haiku about coding.")

high <- chat_groq(
  echo = "none",
  params = params(temperature = 1.2)
)
high$chat("Write a haiku about coding.")
high$chat("Write a haiku about coding.")
```

Exercise: Stable Study Output

Solution

```
prompt <- "Create exactly three flashcards about omitted variable bias. Format as Q: ... A: ..."

chat_low <- chat_groq(
  echo = "none",
  params = params(temperature = 0)
```

```

)

chat_high <- chat_groq(
  echo = "none",
  params = params(temperature = 1)
)

cat(chat_low$chat(prompt))
cat(chat_low$chat(prompt))

cat(chat_high$chat(prompt))
cat(chat_high$chat(prompt))

```

- Recommended for study assistant: `temperature = 0` (more stable outputs).

Switching Providers

- Provider constructor changes, downstream chat interface remains mostly stable.
- Each provider requires corresponding credentials or local runtime.

```

chat <- chat_groq(system_prompt = "You are a helpful assistant.")
chat <- chat_openai(system_prompt = "You are a helpful assistant.")
chat <- chat_anthropic(system_prompt = "You are a helpful assistant.")
chat <- chat_google_gemini(system_prompt = "You are a helpful assistant.")

chat <- chat_ollama(
  model = "llama3.2",
  system_prompt = "You are a helpful assistant."
)

```

Reusable Study Tools

- Section divider matching the slide deck sequence.

Summarizer Design

```

summarize_text <- function(text, detail = "short", audience = "students") {
  system_msg <- paste(
    "You are a study assistant.",
    "Summarize the user's text for", audience, ".",
    "Use a", detail, "summary in bullet points.",
    "Do not add facts that are not in the text."
  )

  chat <- chat_groq(system_prompt = system_msg, echo = "none")
  chat$chat(text)
}

```

Exercise: Add a Constraint

Solution

```
summarize_text <- function(text,
                            detail = "short",
                            audience = "students",
                            max_bullets = 3) {
  system_msg <- paste(
    "You are a study assistant.",
    "Summarize the user's text for", audience, ".",
    "Use a", detail, "summary.",
    "Use at most", max_bullets, "bullet points.",
    "Do not add facts that are not in the text."
  )

  chat <- chat_groq(system_prompt = system_msg, echo = "none")
  chat$chat(text)
}
```

Flashcards and Writing Feedback

```
generate_flashcards <- function(topic, count = 5) {
  system_msg <- paste0(
    "You are a study assistant. Generate exactly ", count,
    " flashcards for the given topic. ",
    "Format each as:\nQ: [question]\nA: [concise answer]\n"
  )

  chat <- chat_groq(system_prompt = system_msg, echo = "none")
  chat$chat(paste("Create flashcards about:", topic))
}

focus_instructions <- list(
  clarity = "Make the text clearer and easier to understand.",
  academic = "Make the text more formal and academic in tone.",
  concise = "Make the text shorter while keeping the key message.",
  grammar = "Fix grammar and spelling errors only."
)

improve_writing <- function(text, focus = "clarity") {
  focus <- match.arg(focus, names(focus_instructions))
  instruction <- focus_instructions[[focus]]
  system_msg <- paste("You are a writing tutor.", instruction)

  chat <- chat_groq(system_prompt = system_msg, echo = "none")
  chat$chat(text)
}
```

Exercise: Build a Translator

Solution

```
translate_text <- function(text, target_language = "German") {
  system_msg <- paste(
    "You are a precise academic translator.",
    "Translate the user's text to", target_language, ".",
    "Preserve technical terms.",
    "Return only the translation."
  )

  chat <- chat_groq(system_prompt = system_msg, echo = "none")
  chat$chat(text)
}

cat(translate_text("Ordinary least squares estimates a linear conditional mean."))
```

One General Helper

```
ask_llm <- function(user_message,
                    system_message = "You are a helpful assistant.",
                    temperature = 0.5) {
  chat <- chat_groq(
    system_prompt = system_message,
    echo = "none",
    params = params(temperature = temperature)
  )
  chat$chat(user_message)
}
```

Structured Extraction

- Section divider matching the slide deck sequence.

When Text Is Not Enough

- Use `chat_structured()` with explicit schema (`type_object`, `type_array`).
- Returns R objects directly, which are easier to validate/analyze than free text.

```
chat <- chat_openai(echo = "none")

result <- chat$chat_structured(
  "My name is Susan and I'm 13 years old",
  type = type_object(
    name = type_string(),
    age = type_number()
  )
)

str(result)
```

Extract a Data Frame

- Use `type_array(type_object(...))` when input text contains repeated records.

```
prompt <- "
  John Smith: Age 30. Income $55,000. Education: B.A.
  Jane Doe: Age 25. Income $42,000. Education: M.Sc.
  Jose Rodriguez: Age 40. Income $78,000. Education: Ph.D.
"

type_person <- type_object(
  name      = type_string("Full name"),
  age       = type_integer("Age in years"),
  income    = type_number("Annual income in USD"),
  education = type_enum(c("B.A.", "M.Sc.", "Ph.D."), "Highest degree")
)

chat <- chat_openai(echo = "none")
people <- chat$chat_structured(prompt, type = type_array(type_person))
people
```

Exercise: Sentiment Schema

Solution

```
review <- paste(
  "The app is easy to use and the charts are beautiful,",
  "but it crashes whenever I import a large CSV file."
)

type_review <- type_object(
  sentiment = type_enum(c("positive", "negative", "neutral")),
  confidence = type_number("Confidence between 0 and 1"),
  reason = type_string("One short sentence explaining the label")
)

chat <- chat_openai(echo = "none")
chat$chat_structured(review, type = type_review)
```

Exercise: Country Table

Solution

```
country_text <- paste(
  "Germany has a GDP of approximately 4.2 trillion USD and is located in Europe.",
  "Brazil's GDP is about 1.9 trillion USD, in South America."
)

type_country <- type_object(
  country = type_string(),
  gdp_trillion_usd = type_number(),
  continent = type_string()
)

chat <- chat_openai(echo = "none")
```

```
countries <- chat$chat_structured(country_text, type = type_array(type_country))
countries
```

Assemble an App-Like Workflow

- Section divider matching the slide deck sequence.

Study Assistant: Architecture

- Architecture maps to base R control flow: menu, switch, repeat, break.
- Recommended implementation pattern is incremental live-coding: menu first, then branches.

```
show_menu <- function() {
  cat("\n=== Study Assistant ===\n")
  cat("1 - Summarize text\n")
  cat("2 - Generate flashcards\n")
  cat("3 - Improve my writing\n")
  cat("4 - Ask a question\n")
  cat("q - Quit\n")
  readline("Choose an option: ")
}

repeat {
  choice <- show_menu()

  if (choice == "q") {
    cat("Happy studying!\n")
    break
  }

  switch(choice,
    "1" = {
      text <- readline("Enter text to summarize: ")
      cat("\nSummary:\n")
      cat(ask_llm(text, "Summarize in clear bullet points.", temperature = 0.3))
    },
    "2" = {
      topic <- readline("Enter a topic: ")
      count <- readline("How many flashcards? (default: 5) ")
      if (count == "") count <- "5"
      sys_msg <- paste0("Generate ", count, " flashcards. Format: Q: ... / A: ...")
      cat(ask_llm(topic, sys_msg, temperature = 0.5))
    },
    "3" = {
      text <- readline("Enter text to improve: ")
      style <- readline("Focus (clarity/academic/concise/grammar): ")
      if (style == "") style <- "clarity"
      sys_msg <- paste("Improve this text. Focus on", style, ". Show improved version.")
      cat(ask_llm(text, sys_msg, temperature = 0.3))
    },
    "4" = {
      question <- readline("Your question: ")
      cat(ask_llm(question))
    }
  )
}
```

```

    },
    cat("Invalid option. Try again.\n")
  )
}

```

Exercise: Add One Menu Item

Solution

```

show_menu <- function() {
  cat("\n=== Study Assistant ===\n")
  cat("1 - Summarize text\n")
  cat("2 - Generate flashcards\n")
  cat("3 - Improve my writing\n")
  cat("4 - Ask a question\n")
  cat("5 - Translate text\n")
  cat("q - Quit\n")
  readline("Choose an option: ")
}

```

Insert this branch inside `switch(choice, ...)`:

```

"5" = {
  text <- readline("Enter text to translate: ")
  lang <- readline("Target language (default: German): ")
  if (lang == "") lang <- "German"
  cat(translate_text(text, target_language = lang))
}

```

Error Handling for API Calls

- Convert raw provider errors into user-friendly diagnostics for common failure classes.

```

ask_llm_safe <- function(user_message,
                          system_message = "You are a helpful assistant.") {
  tryCatch(
    {
      chat <- chat_groq(system_prompt = system_message, echo = "none")
      chat$chat(user_message)
    },
    error = function(e) {
      msg <- conditionMessage(e)
      if (grepl("401|authentication|unauthorized", msg, ignore.case = TRUE)) {
        "Error: Invalid API key. Check your .Renviron file."
      } else if (grepl("429|rate.limit", msg, ignore.case = TRUE)) {
        "Error: Too many requests. Wait a moment and try again."
      } else if (grepl("connection|network|timeout", msg, ignore.case = TRUE)) {
        "Error: Cannot reach the API. Check your internet connection."
      } else {
        paste("Error:", msg)
      }
    }
  )
}

```

Best Practices

- Prompting: role, audience, format contract, hard constraints, edge-case checks.
- Engineering: keep keys out of code, capture responses for scripts, inspect usage costs, prefer low temperature for extraction, review outputs before downstream use.

What We Learned

- Concepts: APIs as remote calls, chat memory and history, system prompts, provider/parameter choices, multimodal input.
- Tools: prompt helper, summarizer, flashcards, translator, structured extraction schemas, study assistant skeleton.